Listas, Dicionários e Registos

Fundamentos de Programação

Bernardo Toninho
IST and INESC-ID

Fundamentos de Programação
2026–2027

Baseado em slides de Luís Caires

Outline

1. Dos Tuplos às Colecções
2. Listas Mutáveis
3. Dicionários: Chaves e Valores
4. Registos com `dataclass`
5. Aprofundamento
6. Síntese

Da Aula 7 para a Aula 8

Na Aula 7

- agrupámos componentes em tuplos;
- definimos invariantes;
- separámos interface e representação;
- construímos valores imutáveis.

Nesta aula

- mantemos colecções que podem crescer;
- associamos chaves a valores;
- damos nomes aos campos de um registo;
- combinamos estas estruturas numa agenda.

Quando um Tuplo Já Não Basta

Um ponto tem sempre duas coordenadas. Uma agenda, pelo contrário:

- pode começar vazia;
- recebe e perde contactos;
- guarda um número variável de entradas;
- deve encontrar rapidamente um contacto pelo nome.

Duas necessidades

Precisamos de colecções mutáveis e de uma forma de localizar valores através de uma chave com significado.

Outline

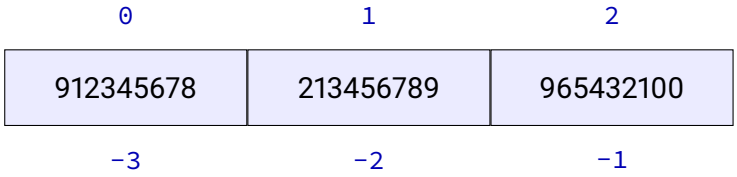
1. Dos Tuplos às Colecções
2. Listas Mutáveis
3. Dicionários: Chaves e Valores
4. Registos com `dataclass`
5. Aprofundamento
6. Síntese

Uma Lista é uma Colecção Ordenada

```
1 numeros = [912345678, 213456789, 965432100]
2
3 print(numeros[0])   # 912345678
4 print(numeros[-1])  # 965432100
5 print(len(numeros)) # 3
```

- Os elementos mantêm uma ordem.
- Os índices começam em `0`; índices negativos contam a partir do fim.
- Ao contrário de um tuplo, a lista pode ser modificada.

Índices e Posições



Contrato da indexação

Para uma lista com comprimento n, os índices não negativos válidos vão de `0` a `n - 1`.

Modificar uma Lista

```
1 numeros = [912345678, 213456789]
2
3 numeros[1] = 210000000
4 numeros.append(965432100)
5
6 print(numeros)
7 # [912345678, 210000000, 965432100]
```

- A afectação substitui um elemento existente.
- `append` acrescenta um elemento no fim.
- As duas operações alteram a própria lista.

Inserir e Remover

```
1 valores = [10, 30]
2
3 valores.insert(1, 20) # [10, 20, 30]
4 ultimo = valores.pop() # devolve 30
5 valores.remove(10)     # remove a primeira ocorrencia
6
7 print(valores)         # [20]
```

- `insert(i, x)` insere `x` antes da posição `i`.
- `pop()` remove e devolve o último elemento.
- `remove(x)` remove a primeira ocorrência de `x`.

Procurar e Contar

```
1 etiquetas = ["casa", "trabalho"]
2
3 print("casa" in etiquetas)     # True
4 print("movel" not in etiquetas) # True
5 print(etiquetas.index("trabalho")) # 1
6 print(etiquetas.count("casa"))     # 1
```

Antes de usar `index`

Convém confirmar a pertença: `index` produz `ValueError` quando o valor não existe.

Previsão: Acompanhar as Alterações

Sem executar, determine o valor apresentado e o conteúdo final de `xs`:

```
1 xs = [10, 20]
2 xs.append(30)
3 xs.insert(1, 15)
4 x = xs.pop()
5 xs.remove(10)
6
7 print(x)
8 print(xs)
```

Resultado Esperado: Alterações à Lista

Problema: seguir a ordem das operações mutáveis.

```
30
[15, 20]
```

1. `append` e `insert` produzem `[10, 15, 20, 30]`.
2. `pop` remove e devolve `30`.
3. `remove(10)` deixa `[15, 20]`.

Encapsular uma Regra sobre a Lista

Queremos impedir números repetidos:

```
1 def adiciona_sem_repetir(xs: list[int],
2                          valor: int) -> None:
3     if valor not in xs:
4         xs.append(valor)
```

- A lista recebida é modificada no lugar.
- O resultado `None` indica que a função não constrói outra lista.
- A regra de unicidade fica concentrada numa operação com nome.

Uma Função Pode Alterar a Lista Recebida

```
1 numeros = [912345678]
2 adiciona_sem_repetir(numeros, 965432100)
3 adiciona_sem_repetir(numeros, 912345678)
4
5 print(numeros)
6 # [912345678, 965432100]
```

Contrato deve dizer o efeito

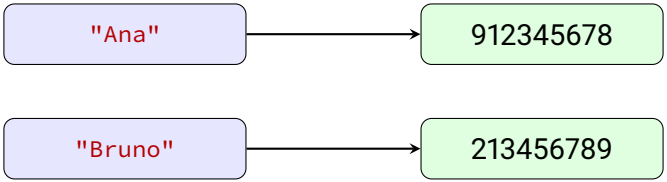
Quando uma função modifica um argumento, esse efeito faz parte do seu contrato e deve ser visível no nome, na documentação e nos testes.

Outline

1. Dos Tuplos às Colecções
2. Listas Mutáveis
3. Dicionários: Chaves e Valores
4. Registos com `dataclass`
5. Aprofundamento
6. Síntese

Da Posição à Chave

Numa lista, encontramos um valor pela sua posição. Numa agenda, queremos usar o nome do contacto:



Dicionário

Uma colecção mutável de associações entre chaves únicas e valores.

Criar e Consultar um Dicionário

```
1 agenda = {
2     "Ana": 912345678,
3     "Bruno": 213456789
4 }
5
6 print(agenda["Ana"]) # 912345678
7 print(len(agenda))    # 2
8 print("Bruno" in agenda) # True
```

- As chaves são únicas.
- `in` testa a presença de uma chave.
- O acesso `agenda[chave]` exige que a chave exista.

Inserir e Actualizar Associações

```
1 agenda = {}
2
3 agenda["Ana"] = 912345678    # inserir
4 agenda["Bruno"] = 213456789  # inserir
5 agenda["Ana"] = 965432100    # actualizar
6
7 print(agenda["Ana"]) # 965432100
```

A mesma sintaxe, dois efeitos

Se a chave não existir, é criada; se já existir, o valor associado é substituído.

Consultar sem Falhar

```
1 agenda = {"Ana": 912345678}
2
3 print(agenda.get("Ana"))            # 912345678
4 print(agenda.get("Carla"))          # None
5 print(agenda.get("Carla", 0))       # 0
```

- `agenda["Carla"]` produziria `KeyError`.
- `get` permite escolher um valor para o caso de ausência.
- O valor por omissão deve ser coerente com o contrato da função cliente.

Remover uma Associação

```
1 agenda = {"Ana": 912345678, "Bruno": 213456789}
2
3 if "Bruno" in agenda:
4     del agenda["Bruno"]
5
6 print(agenda) # {"Ana": 912345678}
```

Pré-condição

`del agenda[chave]` exige que a chave exista; testar a pertença torna essa decisão explícita.

Previsão: Estado do Dicionário

Determine o conteúdo final de `idades` e os valores apresentados:

```
1 idades = {"Ana": 20}
2 idades["Bruno"] = 19
3 idades["Ana"] = 21
4
5 print(idades.get("Carla", -1))
6 del idades["Bruno"]
7 print(idades)
```

Resultado Esperado: Estado do Dicionário

Problema: distinguir inserção, actualização, consulta e remoção.

```
-1
{"Ana": 21}
```

- A chave `"Ana"` é actualizada para `21`.
- A ausência de `"Carla"` produz o valor por omissão `-1`.
- A associação de `"Bruno"` é removida.

Escolher entre Lista e Dicionário

Necessidade	Lista	Dicionário
ordem dos elementos	natural	secundária
acesso por posição	directo	não aplicável
acesso por chave	procura	directo
chaves únicas	não garante	garante
valores repetidos	permite	permite

Pergunta orientadora

Identificamos o valor pela posição que ocupa ou por uma chave do domínio?

Outline

1. Dos Tuplos às Colecções
2. Listas Mutáveis
3. Dicionários: Chaves e Valores
4. Registos com `dataclass`
5. Aprofundamento
6. Síntese

Um Contacto Tem Campos Diferentes

Um contacto não é apenas uma colecção de valores do mesmo género:

- tem um nome;
- tem vários telefones;
- cada telefone tem uma etiqueta, como `"casa"` ou `"trabalho"`.

Registo

Um valor composto cujas componentes têm papéis diferentes e nomes próprios. Uma `dataclass` permite declarar esses campos directamente.

Declarar os Campos de `Contacto`

```python
from dataclasses import dataclass

@dataclass
class Contacto:
    nome: str
    telefones: dict[str, str]
```

- `nome` identifica o contacto.
- `telefones` associa cada etiqueta ao respectivo número.
- A declaração lista explicitamente todos os campos e os seus tipos.

Construir e Aceder aos Campos

```
1 ana = Contacto("Ana", {"casa": "213456789"})
2
3 print(ana.nome)                    # Ana
4 print(ana.telefones["casa"])    # 213456789
5
6 ana.telefones["movel"] = "912345678"
```

- Os campos são acedidos por nome, não por posição.
- O dicionário guardado em `telefones` continua a ser mutável.
- A estrutura combina um registo com uma colecção de associações.

Uma Agenda de Contactos

```
1 Agenda = dict[str, Contacto]
2
3 agenda: Agenda = {}
4 agenda["Ana"] = Contacto(
5     "Ana", {"casa": "213456789"})
```

Escolhas da representação

- O dicionário exterior localiza o contacto pelo nome.
- A `dataclass` identifica os campos de cada contacto.
- O dicionário interior localiza o número pela etiqueta.

Adicionar um Contacto

```
1 def adiciona_contacto(agenda: Agenda,
2                        nome: str) -> None:
3     assert nome != ""
4     if nome not in agenda:
5         agenda[nome] = Contacto(nome, {})
```

- A chave e o campo `nome` representam a mesma identidade.
- Um contacto novo começa sem telefones.
- Repetir a operação não apaga os telefones já existentes.

Completar a Operação: Adicionar um Telefone

Queremos completar esta operação:

```
1 def adiciona_telefone(agenda: Agenda, nome: str,
2                       etiqueta: str, numero: str) -> None:
3     # completar
```

- Que pré-condições devem ser verificadas sobre os três textos?
- Como confirmamos que o contacto já existe?
- Em que dicionário deve ser criada ou actualizada a associação?
- O que acontece se a etiqueta já existir?

Uma Solução Possível: Adicionar um Telefone

Problema: associar uma etiqueta a um número num contacto existente.

```
1 def adiciona_telefone(agenda: Agenda, nome: str,
2                       etiqueta: str, numero: str) -> None:
3     assert nome in agenda
4     assert nome != ""
5     assert etiqueta != ""
6     assert numero != ""
7     contacto = agenda[nome]
8     contacto.telefones[etiqueta] = numero
```

Uma etiqueta nova é inserida; uma etiqueta existente é actualizada. O dicionário da agenda não precisa de ser reconstruído.

Consultar por Nome e Etiqueta

```
1 def procura_telefone(agenda: Agenda, nome: str,
2                       etiqueta: str) -> str | None:
3     contacto = agenda.get(nome)
4     if contacto is None:
5         return None
6     return contacto.telefones.get(etiqueta)
```

- A primeira consulta pode falhar por ausência do contacto.
- A segunda pode falhar por ausência da etiqueta.
- `None` representa explicitamente qualquer uma dessas ausências.

Remover um Telefone

```
1 def remove_telefone(agenda: Agenda, nome: str,
2                     etiqueta: str) -> bool:
3     contacto = agenda.get(nome)
4     if contacto is None:
5         return False
6     if etiqueta not in contacto.telefones:
7         return False
8     del contacto.telefones[etiqueta]
9     return True
```

O resultado distingue uma remoção efectiva de um pedido que não alterou a agenda.

Invariantes e Testes da Agenda

```
1 agenda: Agenda = {}
2 adiciona_contacto(agenda, "Ana")
3 adiciona_telefone(agenda, "Ana", "casa", "213456789")
4
5 assert agenda["Ana"].nome == "Ana"
6 assert procura_telefone(agenda, "Ana", "casa") == "213456789"
7 assert procura_telefone(agenda, "Ana", "movel") is None
8 assert remove_telefone(agenda, "Ana", "casa")
9 assert not remove_telefone(agenda, "Ana", "casa")
```

Invariantes

Nomes e etiquetas não são vazios; a chave exterior coincide com o campo `nome`; cada etiqueta ocorre no máximo uma vez por contacto.

Da Estrutura à Operação

1. Escolher lista quando a ordem e as posições são centrais.
2. Escolher dicionário quando uma chave identifica cada associação.
3. Usar `dataclass` quando as componentes têm nomes e papéis distintos.
4. Encapsular actualizações em funções com efeitos explícitos.
5. Definir invariantes e testar presenças, ausências e actualizações.

Outline

1. Dos Tuplos às Colecções
2. Listas Mutáveis
3. Dicionários: Chaves e Valores
4. Registos com `dataclass`
5. Aprofundamento
6. Síntese

Pergunta: Duas Variáveis, Uma Lista

Sem executar, determine os dois resultados:

```
1 a = [10, 20]
2 b = a
3 b.append(30)
4
5 print(a)
6 print(a is b)
```

- A afectação `b = a` cria outra lista?
- Em que lista actua `append`?

Resultado Esperado: A Lista é Partilhada

Problema: reconhecer duas referências para a mesma lista mutável.

```
[10, 20, 30]
True
```

`a` e `b` designam a mesma lista. Uma alteração observada através de uma variável também é observada através da outra.

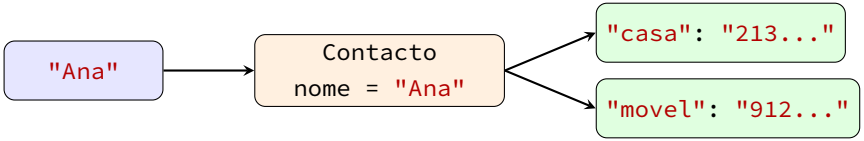
Construir uma Cópia Independente

```
1 a = [10, 20]
2 b = a.copy()
3 b.append(30)
4
5 print(a) # [10, 20]
6 print(b) # [10, 20, 30]
```

Decisão de contrato

Uma função deve deixar claro se modifica a colecção recebida ou se constrói uma nova colecção.

A Estrutura da Agenda



- chave exterior: nome do contacto;
- registo: identidade e campo `telefones`;
- chaves interiores: etiquetas dos números.

Pergunta: Actualizar sem Perder Dados

Considere uma agenda onde Ana já tem o telefone de casa. Queremos acrescentar o telefone móvel.

```
1 agenda["Ana"] = Contacto(
2     "Ana", {"casa": "213456789"})
```

Qual destas estruturas deve ser alterada? Por que razão não devemos substituir o contacto inteiro por `Contacto("Ana", "movel": "912345678")`?

Resultado Esperado: Actualizar o Campo Certo

Problema: acrescentar uma associação preservando os números existentes.

```
1 contacto = agenda["Ana"]
2 contacto.telefones["movel"] = "912345678"
```

Substituir o contacto inteiro apagaria a associação `"casa"`. A actualização deve ocorrer no dicionário `telefones` do registo existente.

Pergunta: Combinar Dois Contactos

Dois registos com o mesmo nome contêm etiquetas diferentes:

- primeiro: `"casa"` e `"movel"`;
- segundo: `"trabalho"` e uma nova versão de `"movel"`.

Defina uma política clara para etiquetas repetidas e explique que dados devem aparecer no resultado.

Uma Solução Possível: Política de Combinação

Problema: combinar telefones, dando prioridade ao segundo contacto.

```
1 def combina_contactos(c1: Contacto,
2                       c2: Contacto) -> Contacto:
3     assert c1.nome == c2.nome
4     telefones = c1.telefones.copy()
5     telefones.update(c2.telefones)
6     return Contacto(c1.nome, telefones)
```

`update` insere etiquetas novas e substitui as repetidas. Os argumentos não são modificados.

Pergunta: Escolher a Representação

Escolha uma estrutura para cada situação e justifique:

- sequência de chamadas pela ordem em que ocorreram;
- contacto encontrado directamente pelo nome;
- nome, correio electrónico e telefone de uma pessoa;
- etiquetas únicas associadas aos telefones de um contacto.

As opções são lista, dicionário e `dataclass`; podem ser combinadas.

Uma Resposta Possível: Representações

Problema: relacionar cada necessidade com a estrutura adequada.

- chamadas por ordem: lista;
- contacto por nome: dicionário;
- campos heterogéneos de uma pessoa: `dataclass`;
- etiqueta para telefone: dicionário.

Uma aplicação real combina frequentemente as três estruturas.

Pergunta: Validar Antes de Actualizar

Que pré-condições deve verificar esta operação?

```
1 def adiciona_telefone(agenda: Agenda, nome: str,
2                       etiqueta: str, numero: str) -> None:
3     # validar e actualizar
```

Considere nomes, etiquetas e números vazios, além da ausência do contacto.

Uma Resposta Possível: Validação

Problema: impedir estados inválidos antes de alterar a agenda.

```
1 def adiciona_telefone(agenda: Agenda, nome: str,
2                       etiqueta: str, numero: str) -> None:
3     assert nome in agenda
4     assert nome != ""
5     assert etiqueta != ""
6     assert numero != ""
7     agenda[nome].telefones[etiqueta] = numero
```

Todas as verificações acontecem antes da actualização, pelo que uma falha não deixa a estrutura parcialmente modificada.

Outline

1. Dos Tuplos às Colecções
2. Listas Mutáveis
3. Dicionários: Chaves e Valores
4. Registos com `dataclass`
5. Aprofundamento
6. Síntese

Conclusão

Hoje vimos

- listas como colecções ordenadas e mutáveis
- dicionários para associar chaves a valores
- registos com campos explícitos através de `dataclass`

A seguir

- ciclos **`for`** e a função **`range`**
- percorrer tuplos e colecções
- excepções e organização em módulos